J Intell Robot Syst (2016) 84:351–369
DOI 10.1007/s10846-016-0358-8

Exploration and Mapping Technique Suited for Visual-features Based Localization of MAVs

Jan Chudoba · Miroslav Kulich · Martin Saska · Tomáš Báča · Libor Přeučil

Received: 15 December 2014 / Accepted: 14 February 2016 / Published online: 15 March 2016
© Springer Science+Business Media Dordrecht 2016

Abstract An approach for long term localization, stabilization, and navigation of micro-aerial vehicles (MAVs) in unknown environment is presented in this paper. The proposed method relies strictly on onboard sensors of employed MAVs and does not require any external positioning system. The core of the method consists in extraction of information from pictures consequently captured using a camera carried by the particular MAV. Visual features are obtained from images of the surface under the MAV, and stored into a map that is represented by these features. The position of the MAV is then obtained through matching with previously stored features. An important part of the proposed system is a novel approach for exploration and mapping of the workspace of robots. This method enables efficient exploring of the unknown environment, while keeping the iteratively built map of features consistent. The proposed algorithm is suitable for mapping of surfaces, both outdoor and indoor, with various density of the image features. The sufficient precision and long term persistence of the method allows its utilization for stabilization of large MAV groups that work in formations with small relative distances between particular vehicles. Numerous experiments with quadrotor helicopters and various numerical simulations have been realized for verification of the entire system and its components.

Keywords MAVs · Visual-features · MAV localization · MAV stabilization · Exploration · Mapping

M. Saska (✉) · T. Báča
Department of Cybernetics, Faculty of Electrical Engineering, Czech Technical University in Prague, Prague, Czech Republic
e-mail: saskam1@fel.cvut.cz

T. Báča
e-mail: bacatoma@fel.cvut.cz

J. Chudoba · M. Kulich · L. Přeučil
Czech Institute of Informatics, Robotics, and Cybernetics, Czech Technical University in Prague, Prague, Czech Republic

J. Chudoba
e-mail: chudoba@labe.felk.cvut.cz

M. Kulich
e-mail: kulich@labe.felk.cvut.cz

L. Přeučil
e-mail: preucil@labe.felk.cvut.cz

1 Introduction

A system for localization of Micro Aerial Vehicles (MAVs) in unknown environments without any artificial landmarks is proposed in this article. The developed algorithm enables stabilization of MAVs based only on onboard sensors and available computational power, and does not require any external localisation (neither global positioning systems, nor motion capture system such as Vicon). The need for such a system is motivated by our endeavour to

allow deployment of MAVs that are flying in compact swarms outside laboratories, and to integrate them in robotic systems being used in both indoor and outdoor applications. Nowadays, inertial measurement units of MAVs may provide sufficient sensory inputs for short-term stabilization and control, however they are not able to stabilize MAVs on their desired positions in long-term missions due to sensor drift and external disturbances.

The presented approach is based on searching for visual features in images obtained from down-looking camera carried by the MAV. During the flight, new captured features are consequently linked relatively to already obtained features by estimation of shift between MAV positions where different images were captured. The MAV movement estimation between capturing of two subsequent images is obtained by fusion of information from inertial measurement unit and statistically measured drift of features being observed in both images. These method may be then used for the MAV position stabilization and for global navigation in metric coordinate frame, providing long-term operability.

The second important part of the proposed system is an algorithm for exploration of the unknown environment, and for building a consistent map from obtained features that are relatively localized. The method is suited for the proposed presented localization using features in the following aspects. It takes into consideration that in general outdoor as well as indoor environments some locations contain more features than others. It means that during flying above some surfaces, it is difficult to find sufficient number of features for the localization (see experimental part of this paper for examples of these surfaces). These areas have to be visited repeatedly, since it was experimentally verified that the set of observed features can be significantly extended during these additional flights. The possibility of finding more features in the same region is enabled due to oscillations of MAV movement, which are usually ineligible, but always occur (mainly in large swarms, where MAVs are negatively influenced by air-flow from propellers of neighbours). MAVs repeatedly visiting the same region newer follow the same trajectory, and images are obtained in slightly different locations and from different angles due to changing MAV tilt. Even in locations with low density of features in captured images, the proposed approach may find sufficient number of features for MAV localization and position stabilization, if the amount of features stored in the map is increased by the repeated mapping.

In addition, the proposed exploration approach takes into account that during flying in areas with insufficient amount of detected features, the localization error increases due to odometry employed for estimation of relative drift between the images. Therefore, the exploration algorithm cannot lead the MAV to outlying frontiers, as it is common in classical approaches. A motion planning algorithm has to be included into the exploration and mapping technique, to ensure that the MAV will return to sufficiently mapped area before the localization error surpasses a given threshold and the consistency of the map would be broken.

The proposed approach to the visual localization provides precision in orders of tens of centimeters. This is sufficient for using with multi-MAV system flying in compact swarms, which is our target application. The long term persistence and the high precision predestine this system for using in applications, in which the current open access systems (such as GPS) do not provide sufficient accuracy and where precise motion capture systems are not available.

2 State of the art and progress beyond state of the art

2.1 Localization, Stabilization, and Control of MAVs

Assuming a MAV has a proprietary attitude control for inclination stabilization, its velocity and position may be stabilized by a control loop including exteroceptive sensor data measurements. Such position or velocity related information may be provided either by artificial external infrastructure for the position measurement, or by sensor mapping the environment characteristics of the MAV surroundings. In outdoor environments the GPS system is most common source of the position in absolute coordinate system [8, 28, 31]. The usage of the GPS for precise MAV localization and navigation is restricted to open spaces, where is good direct visibility to enough satellites. Ideal position precision of common GPS receiver is about 1 meter, which is suitable for most applications, however the error rises when a significant part of sky is occluded, near structures causing receiving of

reflected signals, under the trees or in presence of solar flares affecting signal propagation in atmosphere.

Local external position tracking systems are available, suitable for localization in restricted space. Localization in precision of millimeters with about 100 measurements per second is possible with commercial motion capture systems, see for example [1]. Other visual localization systems offer lower precision and measurement rate in exchange for significantly lower price. Example of such system is described in [22]. The local environment coverage is the most serious disadvantage of these systems, rendering them unusable for most practical applications.

When a MAV has to localize or stabilize itself without aid of external infrastructure, the vision-based methods processing images from MAV-mounted camera are most appropriate, considering cost of sensor and good environment coverage. There are several camera configurations used. Some approaches use a stereo camera [9, 19], which provides raw distance measurements to the objects near the vehicle. As the stereo camera image processing requires high computational resources, many systems use bearing-only approaches using single camera only [6, 10]. Some comparisons between stereo and mono-camera approaches are provided in [25]. Camera mountings on MAV vary from down-looking through tilted to horizontally mounted cameras, each suitable for different type of environment and mode of MAV operation.

Most methods for a visual MAV navigation may be divided to two classes of visual odometry methods and simultaneous localization and mapping (SLAM) methods. Simple visual odometry estimates position changes between two measurements (actual velocity vector) from the optical flow analysis [2, 15] or by visual feature tracking [10, 19]. Position may be estimated by an integration of the velocity. Embedded solutions, such as PX4flow sensor [18], provide visual odometry without need of additional data processing. Non-commercial solution of FPGA image processing for MAV stabilization is provided in [13]. SLAM approaches attempt to suppress the greatest disadvantage of visual odometry methods, which consists in an position error cumulating in time [6]. Building of a persistent map of visual features found in the measured images allows to keep the position error bounded, as long as the MAV position is bounded and map stays consistent. As the MAV is moving through the environment, the position error with respect to metric coordinate frame rises, which is natural property of the SLAM, unless loop-closing mechanism is not used (e.g. in [25]). Some approaches use externally provided geo-referenced image sets for increased robustness and localization in global frame of reference [11].

Most feature-based methods, both visual odometry and SLAM, compute a position change by estimating homography between corresponding current-image-features and past-observed feature set [7, 30, 33]. Some methods overcome homography computation by a direct mapping between optical flow to position [16].

Another navigation method is proposed in [24], estimating position and bearing only along a previously learned trajectory, using a single forward-looking camera.

Different implementations also vary in computational requirements, which is important for practical application on autonomous MAV. Many past articles publish systems performing image processing on external computer, needing sufficient video data link from MAV, or processing data offline from stored images.

The proposed navigation system estimates MAV position changes by comparison of subsequent camera images of ground and by estimating the transformation between these images as the vehicle moves. The results of this transformation (the estimated MAV drift) is then used in feedback to control the vehicle position (to compensate the drift if the MAV is hovering). For robust and effective transformation estimation, the matching of image features, which are invariant to scale and rotation, is used. From the available image features, the speeded-up robust features (SURF, [4]) are used in the experiments described in this article. SURF technique provides good matching robustness compared to computational efficiency, but most of the image features that are used nowadays may be applied in this method with sufficient result.

2.2 Algorithms for Exploration and Mapping

Various exploration strategies have been proposed during last twenty years, which differ in the way how candidates for the next goal are generated and in the criterion how the best candidate is selected. A seminal work of Yamauchi [32] presents an algorithm navigating a mobile robot to the nearest boundary between

an unexplored space and space that has been already recognized as obstacle-free (this boundary is call *frontier*). Several works [20, 21] showed experimentally that even this greedy strategy gives reasonably short trajectories for graph-like environments with upper bound $O(|V|\log(|V|))$, where $|V|$ is the number of vertices of the graph.

Several heuristics improving Yamauchi's original approach has been introduced recently. For example two improvements are discussed in [17] The first improvement for office-like environments utilizes Voronoi diagrams, for which the idea is to explore a whole room at once when a robots enters it first time. The second improvement comes from the fact that a robot is perceiving the environment while it is traversing the planned path and thus the currently approached goal does not necessarily stay a frontier till reaching it. Therefore repetitive checks are performed and when the goal is not a frontier any more, determination of a new goal is invoked.

Other approaches combine several criteria to select the best frontier. For example, authors in [14] where measure an area $A(q)$ of an unexplored part of the environment, which is potentially visible from the candidate q, and combine it with a length of a path to the goal $L(q)$ to get the overall utility of q:

$$g(q) = A(q)e^{-\lambda L(q)},$$

where λ is a positive constant.

Combination of a path length to the goal with expected information gain (determined as an entropy change after performing the action) is introduced in [29], while strategy mixing the path length with information gathered (based on the relative entropy) is introduced in [3] together, who presents solid mathematical framework how to combine several criteria.

Authors in [27] do not use frontier cells directly. Instead, they sample points in a neighbourhood of each frontier and filter then according to predefined criteria. The frontier for which most samples passed the filter is then selected as the next goal. Several authors define *the localization utility* of a cell, which expresses ability of the robot to localize in the cell and integrate it into overall goal-selection criteria, see e.g. [26].

The aforementioned approaches evaluate the distance cost simply as the length of the trajectory from the current robot position to the next goal position. Such defined cost prefers candidates close to the robot without considering subsequent actions. In this paper we present more sophisticated method that takes into account that localization error grows in time as a MAV moves in the environment and the MAV has thus to visit places where it can fully localize in order to limit localization error.

3 Localization Algorithm Description

The basic idea of the proposed MAV visual localization lies in an estimation of parameters of geometric transformation between subsequent images captured from the onboard down-looking camera. In the theoretical description of the method, we assume a flat and horizontal surface under the MAV, but the method is robust enough to deal with terrain undulations. The stabilization of MAV flying above a rough terrain is achieved as shown in the presented outdoor experiments, but its localization (the visual odometry) may be affected by an additional cumulative localization error.

In addition to the transformation between subsequent images, the transformation between current image and a visual map, which is built during the flight from previous observations, is estimated for achievement of long-term stability and robustness. In the mapping part of the algorithm, the observed scale-invariant features are stored in a map. The map of features called $\mathbf{F_m}$ defines the localization frame of reference. Each feature has a 2-dimensional position in a map, which is obtained as the projection of its transformed image coordinates. Additionally, two numbers are attached to each feature for discarding useless features from map: 1) number of repeated observations and 2) time of the first observation.

In the proposed approach, each single camera image is processed as described in this section. The output of the algorithm is the actual map $\mathbf{F_m}$ and the transformation matrix $\mathbf{T}$ in form

$$\mathbf{T} = \left[\, \mathbf{R}\ \mathbf{t} \,\right], \tag{1}$$

where $\mathbf{R}$ is a matrix representing rotation and scale and $\mathbf{t}$ is a translation vector. When new image from camera is captured, the set of features $\mathbf{F_i}$ is detected in the image and feature descriptors are computed. For the MAV position tracking, it is advantageous to have features uniformly distributed in the captured image. This is achieved by an approach inspired by [25],

where the image is divided to sectors and features are detected in each sector separatelly. There is also an adaptive threshold of feature detector set in each sector, controlling number of detected features in optimal range with respect to robustness and computational cost. These features are matched to features stored in the map $\mathbf{F_m}$. We expect that position change between two consecutive images is bounded and therefore all matches can be validated using the last known transformation. In the validity test, the matches with the projection of image feature farther from the matched feature in the map than a adaptive threshold α_1 are discarded. The α_1 threshold is calculated as average projection error over all matches multiplied by constant 3. This mechanism effectively helps to eliminate many accidental wrong matches and so improves method robustness.

The transformation of displacement between images (and consequently displacement in MAV position) is computed from the matching set, which contains two sets of image features $\mathbf{f_i}$ and map features $\mathbf{f_m}$. The proposed method is based on corresponding point set registration algorithm described in [5]. First, centroids of each set ($\mathbf{c_i}$ and $\mathbf{c_m}$) is computed as

$$\mathbf{c_i} = \frac{1}{N}\sum_{n}^{N}{\mathbf{F_i}}^n, \quad \mathbf{c_m} = \frac{1}{N}\sum_{n}^{N}{\mathbf{F_m}}^n, \tag{2}$$

where N is number of matches.

For estimation of the rotation between two feature sets, the feature image coordinates $\mathbf{F_i}$ and $\mathbf{F_m}$ are translated by values $\mathbf{c_i}$ and $\mathbf{c_m}$, respectively (to have centroids of both sets in origin). The rotation matrix is then calculated from a singular value decomposition of matrix

$$\mathbf{H} = \sum_{n}^{N}(\mathbf{F_i}^n - \mathbf{c_i}) \cdot (\mathbf{F_m}^n - \mathbf{c_m})^T, \tag{3}$$

where $\mathbf{F_i}^n$ and $\mathbf{F_m}^n$ are column vectors of image coordinates of feature in image or map respectively, as

$$[\mathbf{u}, \mathbf{s}, \mathbf{v}] = \mathrm{svd}(\mathbf{H}), \tag{4}$$

$$\mathbf{R} = \mathbf{v}^T \cdot \mathbf{u}. \tag{5}$$

The first row of R is multiplied by -1 if $\det(\mathbf{R}) < 0$. The scale of the transformation is estimated as a ratio between average distances of features from origin in each set:

$$scale = \frac{\sum_n^N {\mathbf{F_m}}^n}{\sum_n^N {\mathbf{F_i}}^n}. \tag{6}$$

The translation component $\mathbf{t}$ is computed as a difference between position of the centroid $\mathbf{c_m}$ of the set of features $\mathbf{F_m}$ and the centroid $\mathbf{c_i}$ of the set of image features $\mathbf{F_m}$, which is transformed by the obtained rotation matrix:

$$\mathbf{t} = \mathbf{c_m} - scale \cdot \mathbf{R} \cdot \mathbf{c_i}. \tag{7}$$

The resulting transformation matrix is a compound of the calculated rotation, scale and translation components.

The obtained transformation is used for projection of the detected image feature points into the map. Distance from matching map features is used for detection of outlier matches and removing them from the set of stored matches. The outlier matches are removed from the set of found matches and the transformation is re-calculated. At least 3 matches are required for sufficient transformation estimation. If the transformation is considered as valid, the remaining features from $\mathbf{F_i}$, which were not matched with any feature stored in previous iterations of the algorithm, are projected into map coordinates and added to the map for possible matching in future iterations. The validity test depends on the number of valid matches and re-projection error. Accordingly, older map features, which were not matched in few consecutive images, are removed from the map to prevent unnecessary growth of map size.

The estimated transformation describes MAV position in a map in camera image coordinates relative to its initial position and height above surface. The $scale$ has a meaning of ratio between the actual height above the ground and the initial height. Therefore, MAV position in metric units relative to initial position may be computed based on the known camera image calibration parameters and MAV initial height.

The functionality of the above described algorithm is conditioned by a requirement on zero MAV inclination, which means that the camera optical axis has to be perpendicular to the surface below the MAV. This strong requirement can not be satisfied if using moving MAVs and therefore the effect of inclination has to be compensated. The inclination by angle α in one

axis of MAV hovering in height h causes estimated position error

$$\Delta x = h \cdot \tan\alpha. \tag{8}$$

This distance error can be easily subtracted from the estimated position if we suppose the inclination values are known from the MAV inertial sensors.

The algorithm employed for updating the transformation matrix by the processing of the captured image, which is described in this section, is summarized in listing 1.

Algorithm 1 Visual localization algorithm

```
1  find a complete set of image features F_i;
2  foreach feature f_i ∈ F_i do
3      calculate feature descriptors;
4      find the best match f_m ∈ F_m;
5      add (f_i, f_m) to set M;
6  foreach match (f_i, f_m) from M do
7      calculate error between f_m and projection of f_i
       using the last known transformation T;
8      if error > adaptive threshold α_1 then
9          discard match from M;
10 compute new transformation T from sets of matches
   from M;
11 foreach (f_i, f_m) ∈ M do
12     re-project f_i using T;
13     if reprojection error > threshold alpha_2 then
14         discard match pair from M;
15 if some outlier matches were discarded then
16     re-calculate transformation T;
17 foreach remaining features from F_i, which were not
   matched with any feature from F_m do
18     reproject using T, and store them in the map F_m;
19 foreach feature from F_m having no match in
   consecutive images do
20     discard feature from map
```

4 Localization-aware Exploration

The aforementioned approach to SLAM enables to solve complex navigation tasks, which will be not easy or even impossible to deal with without knowledge of MAV's position. For example, quality of a model of the working environment built from scratch during autonomous navigation of the MAV through this environment is heavily dependent on precision of a localization process.

In this chapter, an approach to such a map-building process is introduced, which takes into account different characteristics of particular parts of the environment influencing localizability (i.e., precision of the position determined by a SLAM algorithm) and thus plans MAV's trajectory not only with respect to time needed to build the map but also with effort to keep a position error in acceptable limits.

Specifically, suppose a-priory unknown environment, whose model is to be built in the form of a regular grid, so that each cell of the grid holds a set of salient features detected by the SLAM algorithm. Note that not all features in a cell are detected at once when the cell is visited. Instead, only a portion of features is correctly recognized and incorporated into the map and thus several visits of the particular cell are needed. The number of visits is not known in advance as it depends on a type of a corresponding piece of the environment. The MAV aims to visit each cell of the map so many times it is sufficient for a successful localization in future.

Another requirement goes from behaviour of the SLAM algorithm similar to dead reckoning: as the position error accumulates during time, the MAV can get lost easily when it operates in an unknown environment without loop closing. To prevent such behaviour and guarantee a limited localization error, the MAV must return time to time (after travelling a defined distance) to already fully explored cell, where it can localized itself.

As not all information about the environment is known when the mission starts the task can be defined as the exploration mission, i.e. the process of autonomous navigation of the MAV in an unknown environment in order to provide a map of it. The proposed approach is thus inspired by the current exploration techniques, e.g. [14, 29, 32] and it has similarly to these a greedy nature, see Algorithm 2. The algorithm walks consecutively through particular cells until all are completely explored (line 2). Features in a currently visited cell are determined and the map in the cell is updated accordingly in each exploration step (line 3). If a next goal to be visited is not determined yet, it is done so and the plan how to reach this goal (i.e., a continuous sequence of cells connecting the current MAV's position with the goal) is found (line 6). Before that, supporting structures allowing effective determination of the goal and the optimal path to it are updated, based on current MAV's

position and the map (line 5). The MAV is finally moved to the first cell of the path and this cells is removed from the path (line 7).

Algorithm 2 The structure of localization-aware exploration.

```
1 path P = ∅;
2 while map is not completely explored do
3     update the current cell;
4     if P == ∅ then
5         update planning structures;
6         generate the new goal G and path P towards it;
7     pop the next cell from the path P and move to it;
```

The key part of the algorithm is selection of the next goal and determination of a path towards it. The goal and the path are chosen according to several criteria:

- **Distance:** Selecting near goals leads to a systematic exploration so that it prevents the MAV to go to distant places and then return back to explore "forgotten" cells in the neighbourhood of the already visited cell.
- **Informativeness:** the MAV should traverse a path, which gives most new information. In other words, paths through not yet visited cells are preferred to ones through already visited.
- **Localizability:** the goal should be selected only if it is guaranteed that there exists a short enough path to a cell where the MAV will be fully localized (i.e. fully mapped cell).

To satisfy these criteria, goal-selection can be formulated as the problem of determining a cell, which minimizes the function l_w:

$$goal = \arg\min_{c \in \mathcal{N}} l_w(\mathcal{P}(start, c)),$$

where $\mathcal{N}$ is a set of not fully mapped cells, $\mathcal{P}(v_a, v_b)$ is a path between v_a and v_b, i.e. a sequence of neighbouring cells starting in v_a and finishing in v_b:

$$\mathcal{P}(v_a, v_b) = \left\langle v_a = v_{i_0} v_{i_1} v_{i_2} \dots v_{i_k} = v_b \right\rangle,$$

and l_w is a weighted length of a path:

$$l_w(\mathcal{P}(v_{i_0}, v_{i_k})) = \sum_{j=0 \dots k-1} w_{i_j, i_{j+1}} d(v_{i_j}, v_{i_{j+1}}), \quad (9)$$

where $w_{i_j, i_{j+1}}$ is a weight of the edge $v_{i_j} v_{i_{j+1}}$ and $(v_{i_j}, v_{i_{j+1}})$ its Euclidean distance. We determine $w(a, b)$ as a fraction $\frac{N_{act}}{N_{total}}$, where N_{act} is a number of features seen in the cell b and N_{total} the expected number of features detectable in that cell. Note that among all possible paths $\mathcal{P}(v_a, v_b)$ between v_a and v_b we take a path which minimizes l_w and for which the following inequality holds:

$$l_E^{trav}(v_a) + l_E(v_a, v_b) + l_E^{loc}(v_b) \leq D_{threshold}, \quad (10)$$

where l_E^{trav} is a distance of a path traversed from the recently fully mapped cell, $l_E(v_a, v_b)$ is a length of a path $\mathcal{P}(v_a, v_b)$, $l_E^{loc}(v, b)$ is a distance of a shortest path from v_b to the nearest fully mapped cell, and $D_{threshold}$ is a predefined threshold. All three distances can be computed using Eq. 9 with all weights set to 1. In other words, this requirement guaranties that a length of a path between two fully mapped cells is bellow the given threshold.

Of course, it can happen that no goal and path satisfying Eq. 10 exist. In that case, the MAV immediately goes to the nearest fully mapped cell $v^\star$. Note that the total path length from the recently visited fully mapped cell after reaching $v^\star$ is $D_{threshold}$ at maximum as the MAV went to the current cell in the previous step based on Eq. 10.

Three different modifications of Dijkstra's algorithm [12] with a priority queue are utilized to determine the next goal and the path. The original Dijkstra's algorithm works on a graph $\mathcal{G}$, for which it computes a shortest path from each vertex in the graph to the defined start vertex together with a length of the path, see Algorithm 3.

To do that, the algorithm stores actual values of the length of the shortest path v_{dist} and predecessor of each vertex along this path v_{prev} for each vertex $v \in \mathcal{G}$. The shortest path for the given vertex v can be then easily determined by walking consecutively over predecessors starting in v. In the initialization stage, distances v_{dist} of all the vertices are set to infinity and their predecessor v_{prev} to a fictive vertex v_∞, what means that the shortest path has not been found yet (lines 1-3). The only exception is the starting vertex, whose distance value is set to 0 (line 4). The start vertex is then put into a priority queue, which is sorted according to v_{dist} (line 5).

The algorithm then consecutively takes the vertices from the priority queue (line 7) and for the current vertex u it processes its neighbours (line 8). If u is reached first time with the algorithm, it is updated accordingly (lines 10 and 11) and added into the priority queue. It it is not the case, a tentative distance to the start is

computed (line 14) for each neighbour v and if it is smaller than the current one, v_{dist} is updated to the tentative distance (line 16) and v_{prev} to u (line 17) and v is updated in the priority queue.

Algorithm 3 Dijkstra's algorithm

```
1  foreach vertex v ∈ G do
2      v_dist = ∞;
3      v_prev = v_∞;
4  start_dist = 0;
5  G.add(start, 0) ;
6  while Q ≠ ∅ do
7      u = Q.pop();
8      foreach neighbour v of u do
9          if v_dist == ∞ then
10             v_dist = u_dist + d(u, v);
11             v_prev = u;
12             Q.add(v);
13         else
14             tentative = u_dist + d(u, v);
15             if tentative < v_dist then
16                 v_dist = tentative;
17                 v_prev = u;
18                 Q.update(v, v_dist);
```

The first variant of the modified Dijkstra's algorithm determines the next goal and paths towards it. The modifications are threefold:

- The search for the goal is terminated when the first not fully explored cell is reached by the algorithm. This is done by adding a condition before the for-cycle on the line 8. If the condition is satisfied the current cell is returned as the next goal. It can be useful in some applications that not yet visited cells are preferably chosen. In that case, this can be also incorporated into the condition: if the algorithm reaches already visited cell first, remember it and when there is no non-visited cell then return this cell as the result.
- Only cells satisfying Eq. 10 should be taken account. Therefore, the for-cycle on the line 8 together with the condition from the previous paragraph will be performed only when Eq. 10 holds.
- In order to efficiently compute Eq. 10, $l_E(v_a, v_b)$ should be also stored and updated for each vertex similarly to v_{dist} (note that v_{dist} stores a current value of l_w).

When no goal is found in the first step, a path to the nearest fully mapped cell is to be determined. The modification is similar to the previous case, Eq. 10 is taken into account and the algorithms ends when it reaches the first fully mapped cell. This means that the same criteria are applied as in the previous case, especially, paths through unvisited cells are preferred, if this does not lead to exceeding $D_{threshold}$.

The last variant of Dijkstra's algorithm is utilized for determination of the distance to the nearest fully mapped cell. Given a set of fully mapped cells $\mathcal{F}$, the modification is consists of two steps: setting $v_{dist} = 0$ for all $v \in \mathcal{F}$ on line 4 and adding them into the priority queue, line 5.

While the map is represented as a grid, Dijkstra's algorithm works on a graph. Fortunately, the transformation is straightforward: cells in the grid correspond to vertices in the graph and each two neighbouring cells in the grid form a pair of directed edges of the graph. On the other hand, two graphs are created: a graph with weighted distances for goal selection and computation of a path to a nearest fully mapped cell, while graph with Euclidean distances is needed for determination of distances to nearest fully mapped cells.

5 System Description

5.1 Visual Localization Module

The MAV localization based on image features detection is computed on Intel Atom CPU 330 at 1.6GHz with integrated Nvidia ION VGA and CUDA platform. For the experiments, CMOS VGA camera with 1/3" chip MT9V032 was used, field of view $61° \times 41°$, configured for 640×480 pixels resolution.

The GPU implementation of SURF algorithm [4] was used to detect features in camera images. Hessian threshold in the feature detection algorithm is adaptively adjusted in order to obtain about 100 features in each image depending on the surface under the MAV. The image processing delay is approximately 250 ms, while the position update rate is about 2 updates per second, in case that a transformation is found.

5.2 Experimental Micro-aerial Vehicle

An experimental quad-copter Mikrokopter L4-ME (see Fig. 1) was used for the verification of

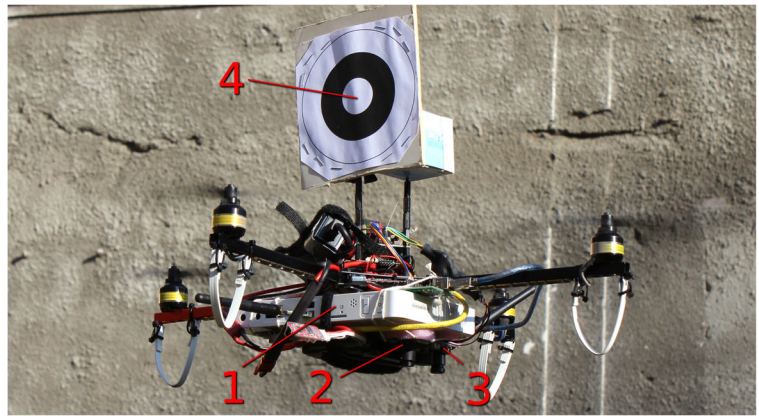


Fig. 1 Mikrokopter equipped with Intel Atom PC (1), localization camera (2), PX4Flow sensor (3) and visual pattern (4) for reference position estimation

the proposed stabilization/localization method. The MAV equipped with above mentioned sensors and computers is able to fly about 5–6 minutes, until a battery is discharged.

This quad-copter, equipped with a FlightCtrl flight control board provides basic MAV stabilization using its inertial sensors. However, this own stabilization naturally suffers from a drift and consequently an additional stabilization system is necessary. We equipped the Mikrokopter MAV with the PX4Flow optical flow sensor [18], pointing its camera downwards.

PX4Flow sensor provides optical flow information (proportional to horizontal velocity) in rates over 100 Hz (max. 250 Hz in good light conditions), which makes it sufficient for the MAV control. Additionally, the PX4Flow sensor is able to measure height above the ground using integrated sonar, which allows us to control MAV altitude.

Feedback control using this sensor enhanced the stabilization significantly, allowing steady hovering for durations of units to tens of seconds, depending on light conditions and ground surface properties. Since the PX4Flow sensor estimates actual velocity of the MAV, the position control is not able to guarantee robust position stabilization in the long term.

The overall control scheme is depicted in Fig. 2. The block (S) stands for the quad-copter, where inputs are requested propeller velocities and the output is a full state of the MAV (3D position, velocity and acceleration). There are basically three nested control loops. The most inner control loop is provided by a proprietary FlightCtrl board (C_1), providing mainly MAV tilt control. Input to the C_1 controller are desired tilt angles (proportional to desired horizontal velocities) and desired vertical velocity. The second control loop provides velocity control by use of PX4Flow (C_2) and proportional-derivative controller. The third control loop includes designed visual feature stabilization method to robustly stabilize MAV position in long term by a controller (C_3), which is realized as a standard PID controller. The possibility to measure real displacement of MAV in a ground-attached coordinate frame enables also to navigate MAV to a requested position with a final error dependent only on a distance from the starting position.

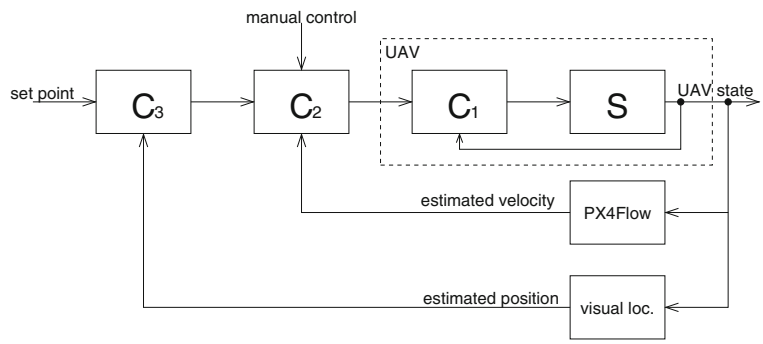


Fig. 2 MAV Controller Scheme

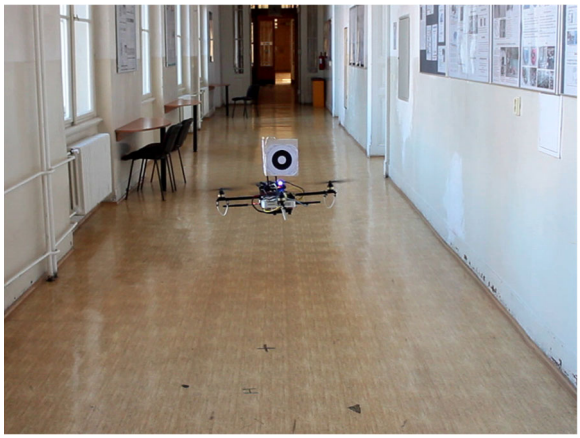


Fig. 3 MAV Hovering Stabilization Experiment

6 Experiments

6.1 Long-term Stability

The proposed method is employed to control and stabilize a position of MAV while hovering in the experiment (see Fig. 3). An external fixed camera was used to obtain a reference position. Beside the reference position and the position estimated by the visual feature localization method, we further show integrated data from the PX4Flow optical flow sensor to prove localization robustness improvement. Visual localization data are used to stabilize MAV position by loop-back control during the experiment.

An information about MAV inclination angles is not used in localization during these experiments, which causes notable oscillations of estimated position and consequently oscillations in real MAV position as a result of loop-back control. This may cause small differences in reference and estimated positions (visible mainly in the time domain), however the result of the experiment is not affected as we want to prove that the average position error is bounded.

A visual-based approach for tracking a circular pattern in a image gathered by an external fixed camera was used as the reference position measurement. Precision of the method in the direction parallel to the image plane is 10 mm (camera resolution is 1920 pixels in one direction and the image covers about 8m of the space) and in the direction perpendicular to the image plane, which is 200 mm in our case. Note also, that the error can be systematically high

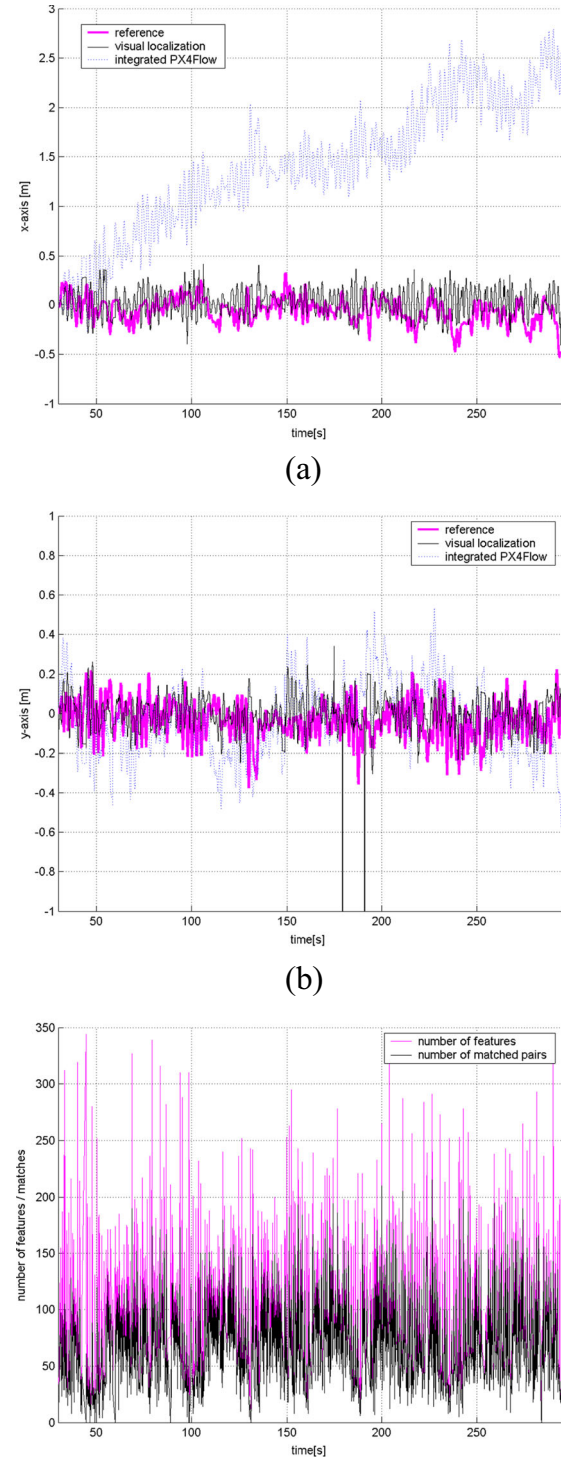


Fig. 4 MAV hovering stabilization in x (**a**) and y (**b**) axes. Blue line – reference, green – the integrated PX4Flow, red – visual localization. **c** The number of detected features and matched pairs

in the case, the camera is not directly facing the pattern. Fortunately, this can be neglected in the presented experiments, as the MAV is relatively far from the localization camera and thus the change of the direction to the pattern is minimal. The detailed description of the localization method can be found in [23].

Figure 4 shows the reference position, the position integrated from PX4Flow measured velocities and the visual feature method position. The duration of the experiment is about 270 seconds, which limited by MAV battery capacity with an actual payload. It is obvious that after few minutes the PX4Flow integrated position estimate drifts from the real position as expected (observed mainly in x-axis in this experiment). The average value of the visual feature position estimator remains bounded. Figure 4 shows the number of detected and matched features during the experiment.



(a)



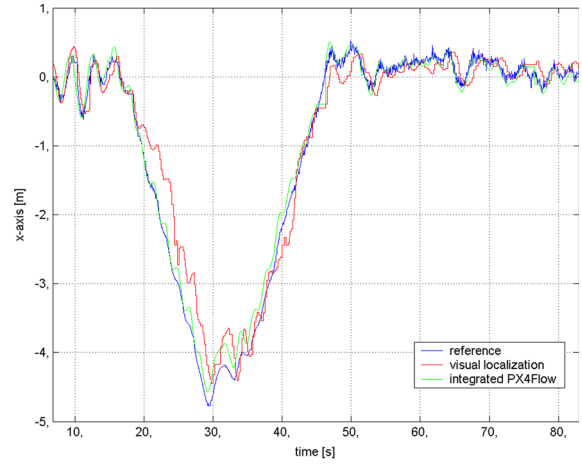
(b)

Fig. 5 MAV navigation by visual feature localization experiment

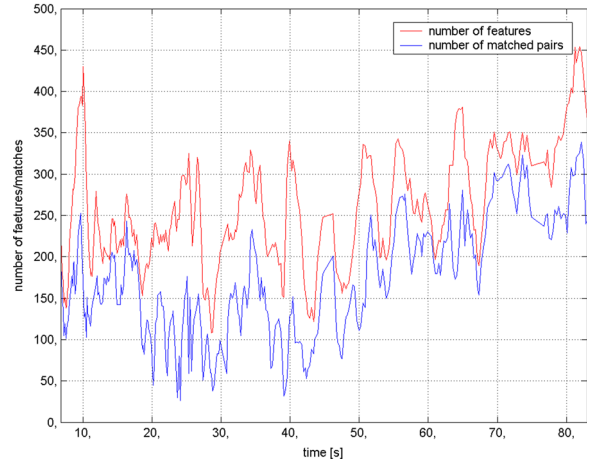
6.2 Navigation Experiment – Short Trajectories

During this experiment a desired position set point is changed to prove ability of the localization method to estimate position while an MAV travels through the previously unknown environment. When the distant position is reached (Fig. 5b), the set point is changed back to the original location (Fig. 5a), so the MAV performs navigation along the previously learned path. Position control is based on the estimated position from visual localization.

Figure 6a shows estimated position compared to externally measured position in one axis. The second



(a)



(b)

Fig. 6 MAV navigation by visual feature localization – x-axis. Blue line – reference, green – integrated PX4Flow, red – visual localization

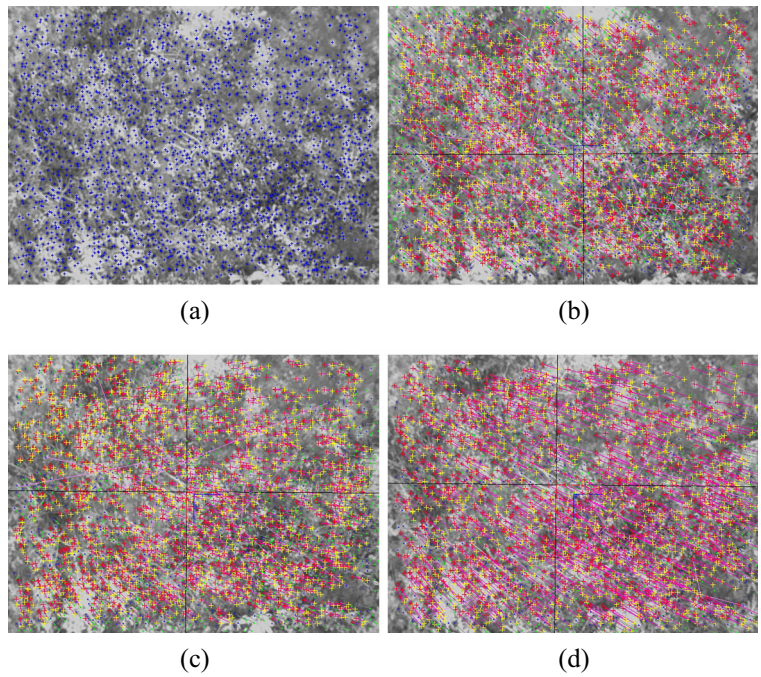


Fig. 7 Image processing – method initialization

axis is not plotted, because the reference visual localization does not provide sufficient precision in this direction. There is a notable position error in far set point (about 4 meters from origin position), compared to the error near the trajectory origin. This error is an consequence of a SLAM nature of the localization method. However, an estimated position of MAV after return to the original location varies about ± 0.2m, which lays in the expected method precision tolerance (including achievable precision of position control). It is evident, that the method is able to navigate MAV to the previously known location with an error bounded by this method precision, when navigating to a location near a previously traveled trajectory. Figure 6b shows the number of detected features and the number of matched features during the experiment. The particular images in Fig. 7 show feature matching during initialization and first steps of the method. A situation when the vehicle returned to the original position is shown in Fig. 8 Lines in the images connect matching feature points in the current and the first (reference) image.

6.3 Navigation Experiment – Long Trajectory

This experiment shows method behavior during navigation along longer trajectory to some distant position and back to the position of the start. At the end of the trajectory a deviation of start-to-end position is compared to the position reported by the visual localization method. More specifically, navigation along the 48 m trajectory (24 m there and back)

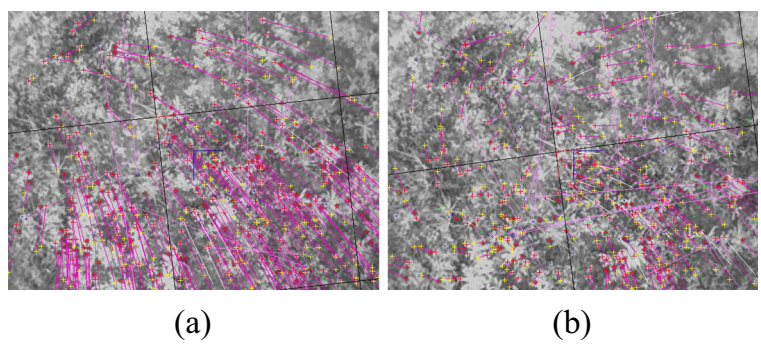


Fig. 8 Image processing – return to the original position

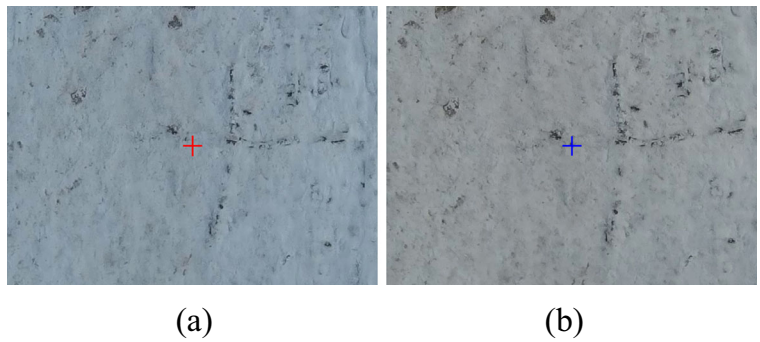


Fig. 9 First and last image frame of the trajectory. The crosses represent the centres of the images, i.e. the MAV position

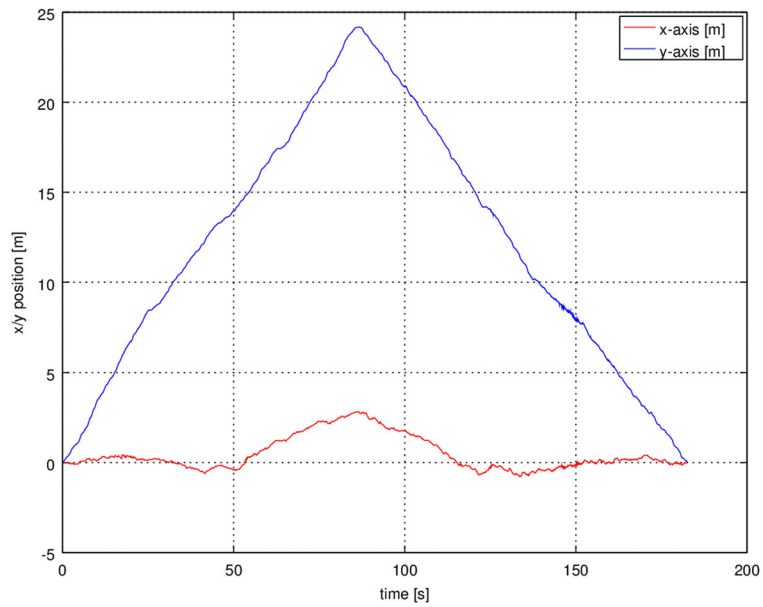


Fig. 10 Position estimation during long trajectory localization experiment

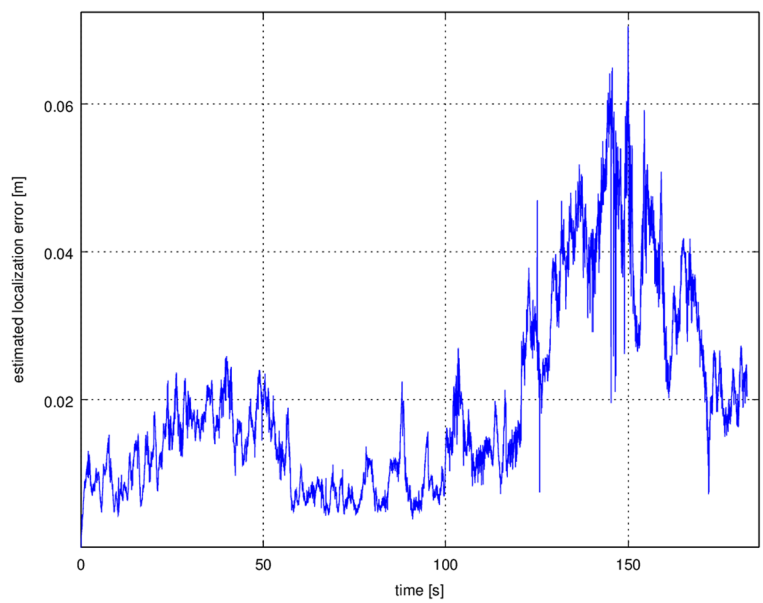


Fig. 11 A position error estimation during the long trajectory localization experiment

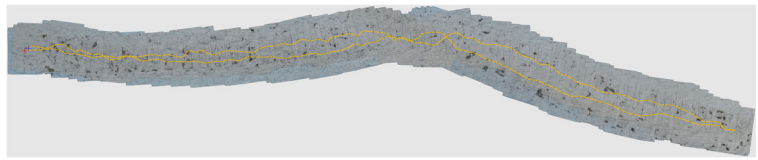


Fig. 12 A composite image of localized camera image with motion trajectory. Red cross marks the start position, blue the destination

approximately 3 m above ground level is performed manually by remote control. The destination position, Fig. 9b, was reached with 56 mm deviation from the starting position, Figure 9a: the start position is at (0, 0), while the real destination position is (50.0, 25.0) [mm].

Figure 10 depicts estimation of MAV position in x/y coordinates separately. The estimated position at the end of the trajectory is (38.5, 55.5) [mm], which means that the absolute localization error at the end position is 33 mm. Estimation of the localization error during the experiment calculated from average visual features matching deviations is shown in Fig. 11. Finally, Fig. 12 shows a composition of camera images placed at estimated positions along the trajectory.

6.4 Terrain Structure Analysis

Different characteristics of various terrain types were analyzed, regarding a number of detected features and percentage of trackable-features. Evaluated terrains are represented by twelve datasets, representative images of each dataset are shown in Fig. 13. Table 1 summarizes average numbers of all detected features and tracked features per dataset. Two approaches were evaluated, the first one with an adaptive detection

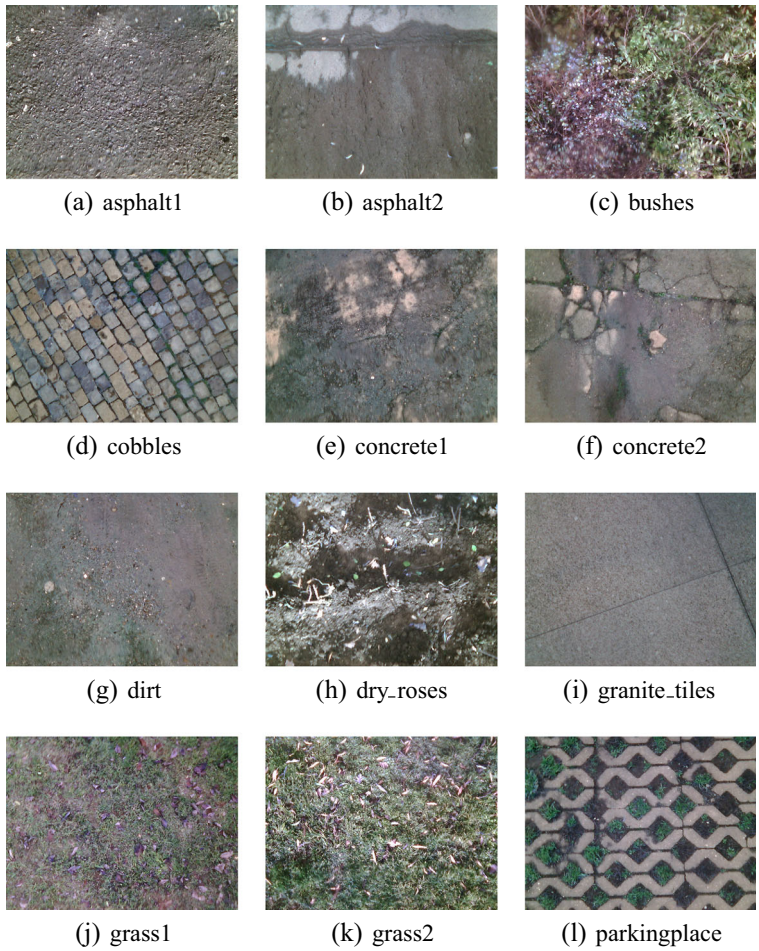


Fig. 13 Terrain analysis dataset – representative images

Table 1 Terrain analysis dataset – numbers of detected and tracked features

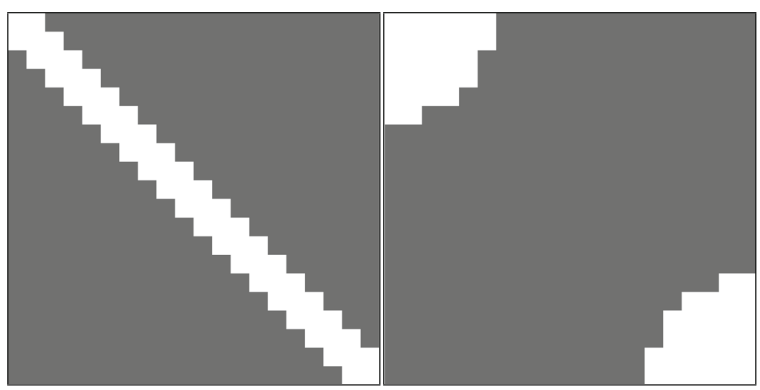
dataset name	auto threshold			fixed threshold		dataset
	tracked	all	%	tracked	all	size
asphalt1	25	90	27,78	109	320	145
asphalt2	26	51	50,98	11	25	429
concrete1	37	52	71,15	60	76	350
concrete2	38	57	66,67	60	77	601
granite_tiles	18	52	34,62	-	-	343
cobbles	40	69	57,97	192	332	490
dirt	30	50	60,00	19	30	182
bushes	46	154	29,87	303	862	217
parkingplace	41	82	50,00	189	386	359
dry_roses	102	173	58,96	589	878	259
grass1	82	117	70,09	392	560	572
grass2	49	170	28,82	414	1167	195

threshold, as described in Section 3. As the total number of features may not be meaningful due to adaptivity, a percentage of tracked features is presented instead. The second approach uses the same fixed threshold for all datasets, showing that terrain surfaces with less details produce less distinctive features. These differences between a number of detected features in different datasets also supports necessity for adaptive threshold setting.

6.5 Localization-aware Exploration

Performance of the presented localization-aware exploration has been evaluated in three different environments in simulation. Note that the only information required by the exploration module is the current MAV position and quality of the map (i.e. localizability of the MAV) at the current position, while the exploration sents a position of the goal to be visited in the next step to the navigation module, which all can be simulated straightforwardly. The only exception is localizability, which we approximate as a number of visits of a particular node in the simulations. Low amount of data to be exchanged between modules also means that the exploration process can be run on a dedicated ground computer in real deployment.

All the environments contain no obstacles and are of size 20×20 cells. The first environment *uniform* contains only cells that has to be visited three times. The second environment *line* is the same, except it contains a diagonal line of cells with high localizability, i.e. cells which it is enough to visit ones,



(a) *line* (b) *islands*

Fig. 14 Environments for exploration evaluation. White pixels represent cells with high localizability, only one visit of these cells is needed to discover all features, while the grey colour stands for the cells to be visited three times

see Fig. 14a, while the last environment contains two islands of cells with high localizability at the opposite corners of the grid, see Fig. 14b.

Three variants of the algorithm were evaluated with different settings of weights in Eq. 9:

- $w^{frac}(a,b) = \frac{N_{act}}{N_{total}}$, where N_{act} is a number of features seen in the cell b and N_{total} is the expected number of features detectable in that cell,
- $w^{unit}(a,b) = 1 \ \ \forall a,b$, i.e. only distance is used to evaluate cells in goal-selection, and
- $w^{mix}(a,b) = 1 + w^{frac}(a,b)$, i.e. combination of the previous weights.

Moreover, another three variants derived from the above ones, which prefer not-yet visited cells were evaluated. In other words, the goal-selection algorithm tries to select not visited cell, which satisfies Eq. 10

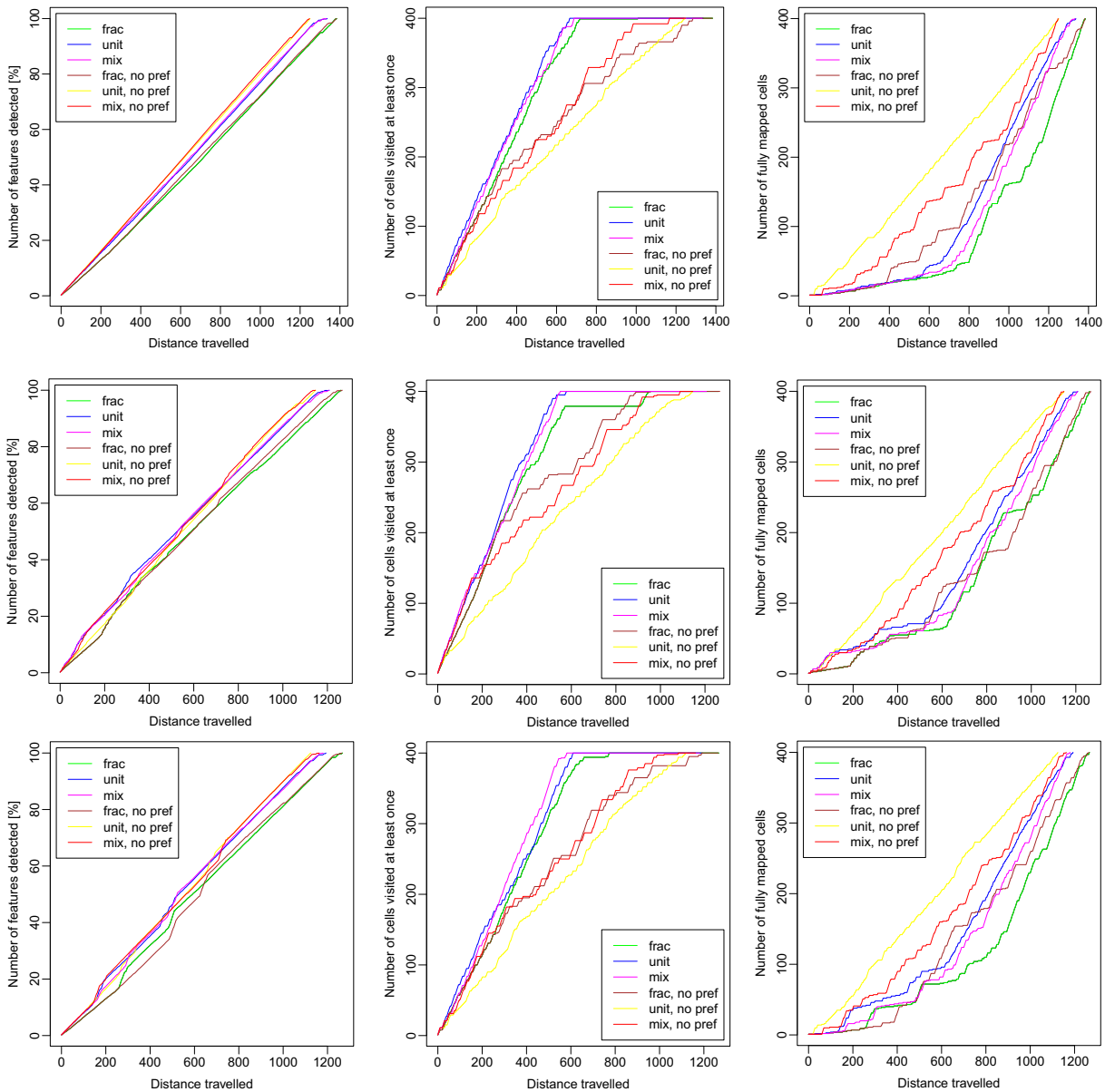


Fig. 15 Evaluation of various variants of the exploration algorithm. Maps (in rows): *empty*, *line*, *islands*. Graphs (in columns) depict progress of (1) percentage of features detected, (2) number of cells visited at leas once, (3) numeber of cells fully mapped.

and only if such a cell does not exist it selects the best, which has been already visited.

All experiments were performed within the same computational environment: a workstation with the Intel®Core i7-3770 CPU at 3.4 Ghz running OS Sabayon with the Linux kernel 3.7.0. The algorithms have been implemented in C++ as a single-thread application. Running times of all algorithms varied from approx. 400 ms (*unit*) to 740 ms (*mix*) for the *uniform* map, where variants with no preference of not visited cells performed worse by 10-20 ms in comparison to the corresponding version with preference. For the *line* and *islands* maps computational times of all the algorithms were better by approx. 10 %.

Comparison of quality of solutions generated by the particular methods is presented in Fig. 15. The first row depicts results for the *uniform* map, the second row results for the *line* map and the third row show results for the *islands* map. The columns consecutively depict progress of (1) the relative number of features detected in the map, the number of cells visited at least once, and (c) the number of fully mapped cells in relation to the distance travelled a MAV.

Differences in progress of percentage of features detected are small as the total exploration time differs by 10 % between the best solutions (*unit* and *mix* without preferences) and the worst ones (*frac* both with and without preferences) for all the maps. Moreover, all the variants give relatively good solutions comparing to the theoretical lower bound, which is 1200 units for the *uniform* map (expecting that every of 400 cells will be visited three times and the distance between neighbouring cells is 1 unit). The best solution found by the *mix* variant with no preferences is 1247.18 units length, which is by 3,9 % worse than the lower bound, the worst solution of *frac* variant with preferences gives 1382.09 units length solution, which is 15 % worse than the optimum. Results for the other two maps are similar.

The situation is different, when we are interested in quickly building an overview of the environment by visiting each cell once and than continue with full mapping. The graphs in the second column show that the variants with preferences to visit unmapped cells first performed much better that these without preference, what is expected behaviour. Both *unit* and *mix* variants with preferences visited all cell twice sooner than the variants without preferences. On the other hand, *unit* without preferences outperformed the other algorithms with respect to a progress in the number of fully mapped cells. While its progress is linear, the progress of the other methods is parabolic-like. This behaviour is beneficial for example for robust localization, in which case the high number of fully mapped cells can be used to refine MAV's position frequently.

Generally, the experiments show feasibility of the proposed method, independently from setting of the weights in the goal-selection process. The produced results for the best setting of the weights are near the theoretical lower bound and reasonable even for the worst cases. Moreover, fine tuning of the weights allows to the exploration problem with various perspectives, either when a quick overview of the environment is required or a high number of fully mapped cells is preferred during exploration.

7 Conclusion

In this article the method for MAV position localization and stabilization, based on visual feature matching, is presented. The method is compared to a method based on optical flow integration using data from the PX4Flow sensor. Performed experiments proved that the method is able to stabilize the MAV on fixed place in long-term operation, in comparison to the inertial or optical flow stabilization methods that suffer from accumulative positioning error.

The localization method has natural limitations caused by necessity to detect distinguishable visual features on a ground under the MAV. The method fails if the visual properties of the ground are uniform, which means that it does not offer sufficient number of features. Another possible problem is caused by similar repeating patterns on the ground, which may result in incorrect feature matching. However, such situation occurs usually only if the MAV is operating in structured urban environments, specifically indoors. In outdoor scenarios the operational conditions of this method are in most cases satisfied, mainly if the MAV is flying in a higher altitude.

Another problem, which requires especial investigation and improvement of the basic method, were observed in MAV movement from "feature-rich" to "feature-poor" environments. The most features are detected in richer part of the environment, since the adaptive feature threshold is used to limit the number

of detected features with the aim of decreasing the computational requirements. Consequently, insufficient number of matches can be found in the "feature-poor" parts, where the MAV is flying to. In the worst case, it may cause a higher position estimation error.

There is a computational limitation caused by available hardware for the visual feature detection and tracking, which limits the image processing to about 4-5 frames per second. Due to this limitation, another faster control loop needs to be employed to locally stabilize the MAV. In the presented experiments, we used underlying controller with the PX4Flow sensor to provide this sufficient stabilization. Nevertheless, utilization of a specialized hardware (e.g. FPGA) for computation of the computational expensive features extraction speeds up the image processing, and the method becomes sufficient also for the low-level stabilization of MAV with only common inertial sensors.

In the current implementation of the proposed method, navigation of the MAV into a previously known place along the unknown path is not supposed. This situation may cause a position mismatch and localization map corruption, which is well known problem in SLAM. In the future development, the loop-closing algorithm needs to be integrated to the method to deal with this issue.

Acknowledgments The work of Jan Chudoba, Miroslav Kulich, and Libor Přeučil has been supported by the Technology Agency of the Czech Republic under the project no. TE01020197 "Centre for Applied Cybernetics". Moreover, the work of Martin Saska was supported by the Grant Agency of the Czech Republic under grant no. P103-12/P756 and the work of Tomáš Báča by CTU grant no. SGS15/157/OHK3/2T/13.

References

1. Vicon motion systems: http://www.vicon.com (2014)
2. Amidi, O.: An Autonomous Vision-Guided Helicopter. PhD thesis, Carnegie Mellon University, Department of Electrical and Computer Engineering Pittsburgh, PA 15213 (1996)
3. Francesco, A., Vincenzo, C.: An information-based exploration strategy for environment mapping with mobile robots. Robot. Auton. Syst. **58**(5), 684–699 (2010)
4. Bay, H., Ess, A., Tuytelaars, T., Van Gool, L.: SURF: Speeded up robust features. Comput. Vis. Image Underst. **110**(3), 346–359 (2008)
5. Besl, P.J., McKay, N.D.: A method for registration of 3-D shapes. IEEE Trans. Pattern Anal. Mach. Intell. **14**, 239–256 (1992)
6. Blosch, M., Weiss, S., Scaramuzza, D., Siegwart, R.: Vision based mav navigation in unknown and unstructured environments. In: 2010 IEEE international conference on robotics and automation (ICRA), pp. 21–28. IEEE (2010)
7. Caballero, F., Merino, L., Ferruz, J., Ollero, A.: Vision-based odometry and slam for medium and high altitude flying uavs. J. Intell. Robot. Syst. **54**(1–3), 137–161 (2009)
8. Caron, F., Duflos, E., Pomorski, D., Vanheeghe, P.: Gps/imu data fusion using multisensor kalman filtering: introduction of contextual aspects. Information Fusion **7**(2), 221–230 (2006)
9. Rodolfo, L., Carrillo, G., Enrique, A., López, D., Lozano, R., Pégard, C.: Combining stereo vision and inertial navigation system for a quad-rotor uav. J. Intell. Robot. Syst. **65**(1–4), 373–387 (2012)
10. Conte, G., Doherty, P.: An integrated uav navigation system based on aerial image matching. IEEE (2008)
11. Conte, G., Doherty, P.: Vision-based unmanned aerial vehicle navigation using geo-referenced information. EURASIP Journal on Advances in Signal Processing, Special section **p1**, 2009 (2009)
12. Dijkstra, E.W.: A note on two problems in connexion with graphs. Numer. Math. **1**(1), 269–271 (1959)
13. Fowers, S.G.: Stabilization and control of a quad-rotor micro-uav using vision sensors (2008)
14. Gonzalez-Banos, H.H., Latombe, J.-C.: Navigation strategies for exploring indoor environments. Int. J. Robot. Res. **21**(10–11), 829–848 (2002)
15. Grabe, V., Bulthoff, H.H., Giordano, P.R.: Robust optical-flow based self-motion estimation for a quadrotor uav. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 2153–2159. IEEE (2012)
16. Guizilini, V., Ramos, F.: Visual odometry learning for unmanned aerial vehicles. In: 2011 IEEE International Conference on Robotics and Automation (ICRA), pp. 6213–6220. IEEE (2011)
17. Holz, D., Basilico, N., Amigoni, F., Behnke, S.: Evaluating the Efficiency of Frontier-Based Exploration Strategies. Munich, Germany (2010)
18. Honegger, D., Meier, L., Tanskanen, P., Marc, P.: An open source and open hardware embedded metric optical flow cmos camera for indoor and outdoor applications. In: IEEE International Conference on Robotics and Automation, Karlsruhe (2013)
19. Kelly, J., Saripalli, S., Sukhatme, G.S.: Combined visual and inertial navigation for an unmanned aerial vehicle. In: Laugier, C., Siegwart, R. (eds.) Field and Service Robotics, volume 42 of Springer Tracts in Advanced Robotics, pp. 255–264. Springer, Berlin Heidelberg (2008)
20. Koenig, S.: Improved analysis of greedy mapping. In: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, Las Vegas, NV, pp. 3251–3257 (2003)
21. Koenig, S., Tovey, C., Halliburton, W.: Greedy mapping of terrain. In: Proceedings of IEEE International Conference on Robotics and Automation, vol. 4, pp. 3594–3599 (2001)
22. Krajník, T., Nitsche, M.: A practical multirobot localization system. J. Intell. Robot. Syst. (2013)
23. Krajník, T., Nitsche, M., Faigl, J., Duckett, T., Mejail, M., Preucil, L.: External localization system for mobile robotics. In: Proceedings of the International Conference on Advanced Robotics. IEEE, Montevideo (2013)

24. Krajník, T., Nitsche, M., Pedre, S., Preucil, L., Mejail, M.: A Simple Visual Navigation System for an UAV. In: International Multi-Conference on Systems, Signals and Devices, p. 34. IEEE, Piscataway (2012)
25. Lemaire, T., Berger, C., Jung, I.l.-K., Lacroix, S.: Vision-based slam: Stereo and monocular approaches. Int. J. Comput. Vis. **74**(3), 343–364 (2007)
26. Makarenko, A.A., Williams, S.B., Bourgault, F., Durrant-Whyte, H.F.: An experiment in integrated exploration. In: IEEE/RSJ International Conference on Intelligent Robots and System, pp. 534–539. IEEE (2002)
27. Newman, P.M., Bosse, M., Leonard, J.J.: Autonomous feature-based exploration. In: IEEE International Conference on Robotics and Automation. Taiwan (2003)
28. Rönnbäck, S.: Developement of a ins/gps navigation loop for an uav. Master's thesis, 81 (2000)
29. Stachniss, C., Grisetti, G., Burgard, W.: Information gain-based exploration using Rao-Blackwellized particle filters. In: Proceedings of Robotics: Science and Systems. Cambridge, MA, USA (2005)
30. Wang, C.-L., Wang, T.-M., Liang, J.-H., Zhang, Y.-C., Yi, Z.: Bearing-only visual slam for small unmanned aerial vehicles in gps-denied environments. Int. J. Autom. Comput. **10**(5), 387–396 (2013)
31. Wendel, J., Meister, O., Schlaile, C., Trommer, G.F.: An integrated gps/mems-imu navigation system for an autonomous helicopter. Aerosp. Sci. Technol. **10**(6), 527–533 (2006)
32. Yamauchi, B.: A frontier-based approach for autonomous exploration. In: Proceedings of IEEE International Symposium on Computational Intelligence in Robotics and Automation, pp. 146–151. IEEE Comput. Soc. Press (1997)
33. Zhao, S., Lin, F., Peng, K., Chen, B.M., Lee, T.H.: Homography-based vision-aided inertial navigation of uavs in unknown environments. In: AIAA Guidance, Navigation, and Control Conference (2012)

Jan Chudoba received his MSc. degree at Czech Technical University in Prague, 2004. Currently, he is a research fellow at Department of Cybernetics. He has >20 publications in robotics. His interests include navigation of mobile robots, mapping and sensor data fusion.

Dr. Miroslav Kulich is currently an assistant professor at the Czech Institute of Informatics, Cybernetics, and Robotics, Czech Technical University (CTU) in Prague. He received the Ph.D. degree in Artificial Intelligence and Biocybernetics at CTU in Prague, and RNDr. degree at Charles University in Prague, Faculty of Mathematics and Physics. He also spent 6 months at research fellowship at Helsinki University of Technology, Automation Technology Laboratory, Finland. Miroslav Kulich is an author or co-author of more than 50 publications at international conferences, 6 articles in impacted journals, 1 book, an editor of European Robotics Symposium (EUROS 2008) proceedings and he received two Best Paper Awards (EMCSR 2010, MESAS 2015). He serves as a reviewer for several impacted journals as well as a program committee member of relevant international conferences. His research interests include planning for single and multi-robot systems, especially in exploration and search & rescue scenarios.

Martin Saska received his MSc. degree at Czech Technical University in Prague, 2005, and his Ph.D. degree at University of Wuerzburg, Germany, within the PhD program of Elite Network of Bavaria, 2009. Since 2009, he is a research fellow at Czech Technical University in Prague, where he founded and heads the Multi-robot Systems lab (http://mrs.felk.cvut.cz/) and co-founded Center for Robotics and Autonomous Systems with more than 40 researchers cooperating in robotics (https://robotics.fel.cvut.cz/cras/). He was a visiting scholar at University of Illinois at Urbana- Champaign, USA in 2008 and at University of Pennsylvania, USA in 2012, 2014 and 2016, where he worked with Vijay Kumar's group within GRASP lab. He is an author or co-author of more than 40 publications in peer-reviewed conferences and more than 10 publications in impacted journals, including IJRR, AURO, RAS, ASC, EJC. He received best paper award at ICUAS 2013 and he was best paper award honored finalist at ICRA 2008. He was program committee member of RA 2007, FLAIRS 27, IDC 2013, FLAIRS 28, ICUAS 2015, ICRA 2016, ICUAS 2016 and serves as reviewer in IEEE trans. on automatic control, IEEE trans. on automation science and engineering, Journal of intell. & robotic syst., Autonomous robots, Swarm intelligence, European journal of control, Journal of the franklin institute, Engineering and applied mathematics. His interests include cooperation and control of multi-robot systems, formation flying, and bio-inspired swarms.

Tomáš Báča is a PhD student at Czech Technical University in Prague, with topic: Model predictive control of groups of unmanned aerial vehicles. His interests include control and stabilization of multirobot systems, and application of aerial vehicles in photography.

Dr. Libor Přeučil obtained his MEE (1986) and Ph.D (1992) in Technical Cybernetics - Robotics and Computer Vision, both from the Czech Technical University in Prague (CTU). Currently, he is an Assistant Professor in Robotics and AI at the Czech Institute of Informatics, Cybernetics, and Robotics, CTU. His international experience comprises stays with the Department of Applied Mathematics, Technical University, Munich, Germany and the France Telecom University, Dept. of Computer Science. Libor Přeučil founded and heads the Inteligent and Mobile Robotics lab at CTU (http://imr.ciirc.cvut.cz) and co-founded the Center for Advanced Field Robotics (http://imr.ciirc.cvut.cz/cafr). He has also been principal (co-) investigator in numerous international projects conducting joint research in the field of UGV and UAV robotics, multi-robot systems, bioinspired robotics, etc. Libor Přeučil is author or co-author of over 200 publications in proceedings of international conferences, journal papers and co-author of three textbooks on AI and 4 edited books. Besides that he is co-chair of IEEE TC on Networked Robots, member IFAC Workgroup for Autonomous Vehicles, organizer and/or member of program committees of relevant conferences (ECMR, ICINCO, MESAS, EUROS, etc.). Currently his primary research interests cover the area of intelligent mobile robotics: namely outdoor robotics, HRI systems, mobile robots for uncontrolled environments and life-long autonomy, all in the UGV and UAV domains - in particular robot navigation, uncertain sensory processing a environment modeling.